```
;
================================================================================
; AVALIAÇÃO: Manipulação de Vetores e Sub-rotinas em AVR Assembly
; Microcontrolador: ATmega328P
;
================================================================================

.include "m328Pdef.inc"

; ------------------------------------------------------------------------------
; PARTE 2 - SEGMENTO DE DADOS (SRAM)
; ------------------------------------------------------------------------------
.DSEG
.ORG SRAM_START

VA: .BYTE 12          ; Vetor A (12 posições de 8 bits)
VR: .BYTE 12          ; Vetor Reverso (12 posições de 8 bits)

; ------------------------------------------------------------------------------
; PARTE 2 - CÓDIGO TESTE PRINCIPAL
; ------------------------------------------------------------------------------
.CSEG
.ORG 0x0000

start:
    ; --- Inicialização Obrigatória do Ponteiro de Pilha (Stack Pointer) ---
    LDI R16, LOW(RAMEND)
    OUT SPL, R16
    LDI R16, HIGH(RAMEND)
    OUT SPH, R16

    ; 1. Inicializar VA com os valores de 31 a 42 (12 elementos)
    LDI XL, LOW(VA)
    LDI XH, HIGH(VA)
    LDI R20, 12       ; Tamanho = 12
    LDI R21, 31       ; Valor Inicial = 31
    RCALL init_array

    ; 2. Procurar o valor 31 no VA (Esperado: R22 = 1, pois é a 1ª posição)
    LDI XL, LOW(VA)
    LDI XH, HIGH(VA)
    LDI R20, 12
    LDI R21, 31
    RCALL search_array

    ; 3. Procurar o valor 5 no VA (Esperado: R22 = 0, pois o valor não existe)
```

```
    LDI XL, LOW(VA)
    LDI XH, HIGH(VA)
    LDI R20, 12
    LDI R21, 5
    RCALL search_array

    ; 4. Copiar o vetor VA para o VR de forma reversa
    LDI XL, LOW(VA)
    LDI XH, HIGH(VA)
    LDI YL, LOW(VR + 12)  ; Y aponta para a última posição + 1 (VR + 12)
    LDI YH, HIGH(VR + 12)
    LDI R20, 12
    RCALL reverse_array

    ; 5. Procurar o valor 31 no VR (Esperado: R22 = 12, pois o 31 foi para o fim)
    LDI XL, LOW(VR)
    LDI XH, HIGH(VR)
    LDI R20, 12
    LDI R21, 31
    RCALL search_array

main_loop:
    RJMP main_loop      ; Trava a execução para análise na SRAM


;
================================================================================
; PARTE 1 - SUB-ROTINAS
;
================================================================================

; ------------------------------------------------------------------------------
; Sub-rotina: init_array
; Descrição: Preenche um vetor com uma sequência de valores a partir de R21.
; Parâmetros: X = Ponteiro inicial | R20 = Tamanho | R21 = Valor Inicial
; ------------------------------------------------------------------------------
init_array:
    ; Preserva os registradores que serão alterados internamente
    PUSH R20
    PUSH R21
    PUSH XL
    PUSH XH

loop_init:
    ST X+, R21          ; Salva R21 no endereço apontado por X e avança X (+1)
    INC R21             ; Próximo valor da sequência (ex: 31 -> 32 -> 33)
```

```
    DEC R20             ; Decrementa o contador de posições
    BRNE loop_init      ; Repete enquanto R20 != 0

    ; Restaura os registradores na ordem inversa
    POP XH
    POP XL
    POP R21
    POP R20
    RET


; ---------------------------------------------------------------------------
; Sub-rotina: reverse_array
; Descrição: Copia os dados do vetor X para Y em ordem inversa.
; Parâmetros: X = Ponteiro Origem | Y = Ponteiro Destino (Última Posição + 1)
;             R20 = Tamanho dos Vetores
; ---------------------------------------------------------------------------
reverse_array:
    PUSH R16            ; Registrador auxiliar para transporte do byte
    PUSH R20
    PUSH XL
    PUSH XH
    PUSH YL
    PUSH YH

loop_reverse:
    LD R16, X+          ; Lê da origem (início ao fim) e avança X (+1)
    ST -Y, R16          ; Decrementa Y (-1) e grava no destino (fim ao início)
    DEC R20             ; Decrementa contador
    BRNE loop_reverse   ; Repete enquanto R20 != 0

    POP YH
    POP YL
    POP XH
    POP XL
    POP R20
    POP R16
    RET


; ---------------------------------------------------------------------------
; Sub-rotina: search_array
; Descrição: Busca a 1ª ocorrência de R21 no vetor X e retorna seu índice (1 a N).
; Parâmetros: X = Ponteiro Origem | R20 = Tamanho | R21 = Valor Procurado
; Retorno:    R22 = Índice (1 a N) ou 0 se não encontrado
; ---------------------------------------------------------------------------
search_array:
    PUSH R16            ; Registrador para leitura do elemento
```

```
    PUSH R20          ; Contador de elementos restantes
    PUSH R23          ; Contador do índice atual (inicia em 1)
    PUSH XL
    PUSH XH

    LDI R22, 0        ; Inicializa o retorno com 0 (não encontrado)
    LDI R23, 1        ; O primeiro elemento é index = 1

loop_search:
    LD R16, X+        ; Lê elemento de X e avança o ponteiro
    CP R16, R21       ; Compara elemento lido com o buscado (R21)
    BREQ found        ; Se forem iguais, pula para "found"

    INC R23           ; Incrementa o índice atual
    DEC R20           ; Decrementa contador de elementos restantes
    BRNE loop_search  ; Se R20 != 0, continua procurando
    RJMP end_search   ; Se terminou o vetor sem achar, vai para o fim

found:
    MOV R22, R23      ; R22 recebe o índice onde foi encontrado

end_search:
    ; IMPORTANTE: R22 NÃO leva POP pois é o registrador de retorno!
    POP XH
    POP XL
    POP R23
    POP R20
    POP R16
    RET
```